A Controlled and Comparative Study of the Effectiveness of Reactive and Planning Strategies of LLM Agents in Navigation Tasks

Changxin Wan **Calvin Glisson** **Kedar Nitin Shiralkar**

Bhargav Srikanta Prasad Hoskote **Vedant Deepak Borkute**

Abstract

The Large Language models (LLMs) trained from massive text datasets allow them to be equipped with sufficient knowledge of different research areas. Recent studies have demonstrated that LLMs can assist vehicle agents in solving complex sequential decision-making tasks such as navigation. However, the performance on navigation tasks varies with different reasoning and prompting methods (e.g., Chain of Thought, Buffer of Thought, etc.) of the LLM agent. In this study, we compare 4 different LLM reasoning methods (i.e., ReAct, Chain of Thought, Tree of Thought, Buffer of Thought) with the Reactive baseline. The MiniGrid environment is selected to test the effectiveness of the 5 methods. More specifically, the average success rate, exploration steps, and total tokens used are applied as the performance metrics. Evaluation was done on three of the MiniGrid environments, including `Empty8×8`, `LavaGapS6` and `LavaCrossingS9N2` while considering the difficulty levels. The experimental results demonstrate that most of the reasoning-enhanced LLM agents outperform the reactive baseline in success rate and exploration steps, with a certain compute trade-off.

1 Introduction

By scaling model capacity and training on massive, diverse text corpora, large language models (LLMs) have demonstrated strong capabilities in language understanding, knowledge generalization, and conversational dialogue [2]. These capabilities have motivated growing interest in deploying LLMs as planning agents that can perceive an environment, reason about possible actions, and make decisions in interactive tasks compared with the traditional deep reinforcement learning (DRL) based method [5]. Among these tasks, navigation provides a useful scenario because it requires an agent to interpret the ego and neighbors states, avoid hazards, maintain short-term memory, and plan towards a goal with a partially observed environment.

Recent studies suggest that LLM-based agents can be effective in sequential decision-making settings, but their performance depends heavily on how reasoning is structured[3, 4]. A purely reactive agent may respond quickly and cheaply, yet struggle in environments that require longer-horizon planning. In contrast, prompting strategies such as Chain of Thought, ReAct, Tree of Thought, and Buffer of Thought may improve decision quality by encouraging explicit reasoning, action evaluation, or memory use. However, these methods also introduce tradeoffs in token cost, inference complexity, and exploration efficiency. As a result, it remains unclear which reasoning strategy is most effective for navigation tasks under different levels of difficulty.

To address this issue, a controlled and comparative study is conducted to identify the effectiveness of different LLM agents, as shown in Figure 1. We deploy 5 kinds of LLM agents in the MiniGrid environment with the aspect of memory and reasoning methods. The effectiveness of the 5 methods is

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

discussed in terms of the performance metrics of average success rate, exploration steps, and tokens used. The contributions of this study are as follows:

- **LLM implementation on navigation.** We design effective prompts and different reasoning methods for the LLM agents to achieve robust decision-making for navigation in the MiniGrid environments.
- **Comparative study on LLM agents.** We evaluate the 5 LLM agents in three environments with increased difficulty levels, then compare their success rate and tokens used for model selection guidance.

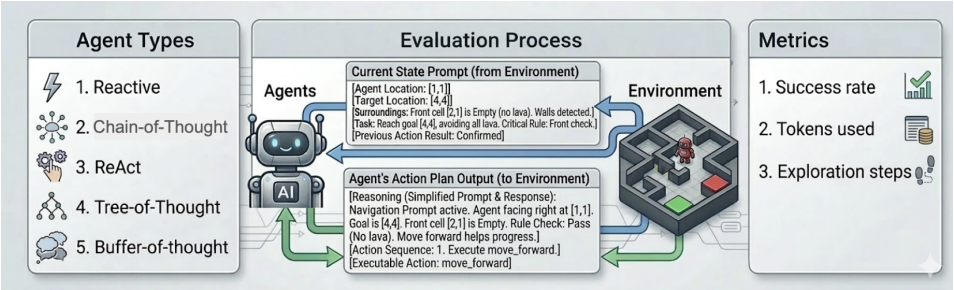


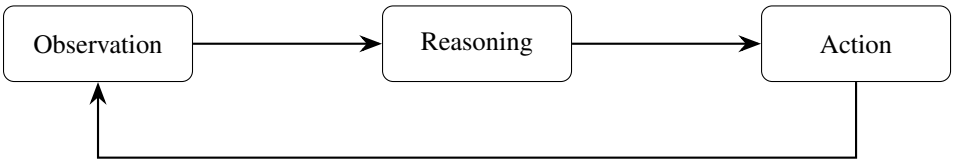
Figure 1: The overall framework for the comparative study

2 Background

2.1 LLM Agents and Reasoning Frameworks

Large Language Models (LLMs) have recently demonstrated strong capabilities beyond traditional natural language processing tasks. Modern LLMs are capable of performing reasoning, planning, and sequential decision-making when provided with structured prompts. This has led to the emergence of **LLM agents**, where language models are placed in an interaction loop with an environment and generate actions based on observations.

Formally, an LLM agent operates within a closed-loop interaction cycle consisting of three stages:



At each timestep, the environment produces an observation describing the current state. This observation is converted into a textual representation and provided to the language model as part of a prompt. The LLM processes the prompt and generates a response that contains the next action to perform. The selected action is then executed in the environment, which updates the state and produces the next observation.

Recent research has shown that **prompting strategies** can significantly influence the reasoning behavior of LLM agents. Instead of directly predicting an answer or action, models can be encouraged to generate intermediate reasoning steps. Techniques such as **Chain-of-Thought (CoT)** prompting demonstrate that explicitly asking the model to reason step-by-step can improve performance on complex tasks. Subsequent work has extended this idea to more sophisticated reasoning frameworks such as **ReAct**, **Tree-of-Thought**, and **Buffer-of-Thought**, which incorporate structured reasoning, branching exploration, and reusable reasoning templates.

These reasoning frameworks are particularly relevant for **sequential decision-making tasks**, where agents must select a sequence of actions while considering environmental constraints. Navigation tasks in grid environments provide a controlled setting to evaluate these reasoning strategies. In such environments, the agent must reach a goal location while avoiding hazards such as walls or lava tiles.

The agent must interpret spatial information, evaluate potential moves, and plan safe paths toward the goal.

In this work, we evaluate different reasoning strategies for LLM agents in navigation tasks using the MiniGrid environment. Specifically, we compare reactive decision-making with reasoning-based approaches including Chain-of-Thought, ReAct, Tree-of-Thought, and Buffer-of-Thought. By evaluating these methods across environments of increasing difficulty, we aim to understand the trade-offs between reasoning complexity, computational cost, and navigation performance.

2.2 MiniGrid environment

MiniGrid[1] is a grid-based environment commonly used in reinforcement learning research, providing tasks where agents learn to achieve simple goals. MiniGrid allows for the adjustment of difficulty through settings such as grid size, goal location, and obstacles, making it suitable for evaluating the performance of various reinforcement learning algorithms. Recent studies have utilized the MiniGrid environment to assess agent navigation and learning performance, proposing methods to extend these capabilities into more complex environments. Figure 2 shows some example Minigrid environments, and Figure 3 shows our chosen environments in order of difficulty.

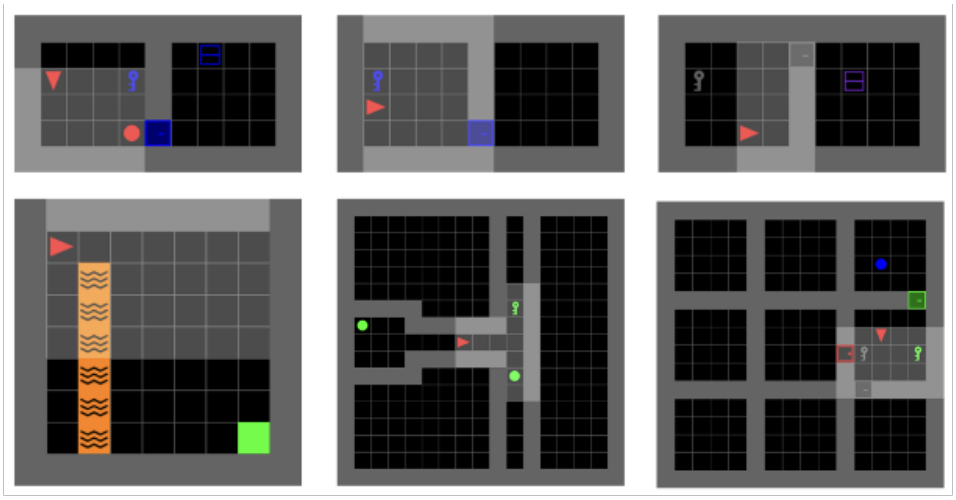


Figure 2: The environments included in MiniGrid

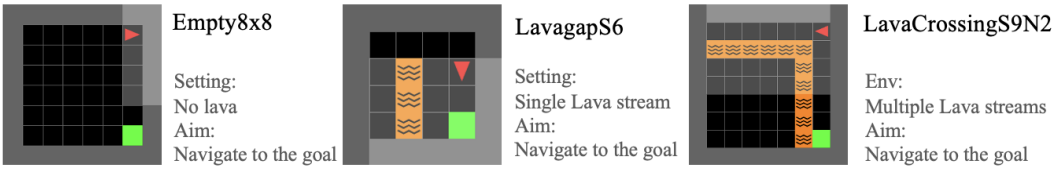


Figure 3: The selected three environments with increasing difficulty

3 Methodology

3.1 Reactive

The reactive agent directly outputs the action without any reasoning or memory of previous observations and actions. The system and user prompt input of the reactive LLM agent is illustrated in Figure 4. The system prompts constraints the reactive to take action directly without any reasoning. The user prompt guides the LLM agent to take action to reach the goal tile based on the current observation. The safety rule (i.e., `Never move_forward if the front cell contains lava`) is applied to the agent to navigate with safety.

System prompt

```
You are a Reactive AI agent in a 2D grid world. You respond ONLY with the exact name
of the action to take. You cannot do anything else except taking action.
```

User prompt (Interaction)

```
Goal: reach the goal tile while avoiding lava.
Important rule: Never move_forward if the front cell contains lava.
Current Status:
        {text_observation}
You should just take one action
Allowed actions: {turn_left, turn_right, move_forward}
Do not output multiple actions.
```

Action output with LLM response

```
text = response.choices[0].message.content.strip()
action = text if text in {turn_left, turn_right, move_forward} else move_forward
```

Figure 4: Prompt of reactive agent

```
Current Position and Direction:
  - Agent at [3,1] facing up.
Nearby box status:
  - Front: wall,
  - Left: empty,
  - Right: lava,
  - Behind: empty.
  - The cell directly in front of you contains wall.
Goal status:
  - Goal at [7,7]. Distance: 10 steps.
  - You need to go: right, down.
  - You are NOT facing the goal - consider turning.
Lava Position:
  - [1,4], [3,4], [4,1], [4,2], [4,3], [4,4], [4,5],
    [4,6], [5,4], [6,4], [7,4].
```

Figure 5: An example of the text observation

The text observation should consider the ego agent state (i.e., the position and direction), neighbors box states, the goal and lava position. A example of text observation is as follows:

3.2 Chain of Thought

Chain of Thought (CoT) [8] is a simple but effective prompt engineering technique that instructs the model to output step-by-step reasoning before deciding upon its action. For our implementation of CoT, we instruct the model to "`Output a careful step-by-step analysis of what the rules are, what has happened in the past states and actions, what actions are valid, and what would be the best strategy to take.`" In our tests, this leads to outputs that first analyze the current position/orientation, goal direction, candidate actions, and overall strategy before deciding upon an action.

3.3 ReAct

ReAct (Reason + Act) [11] integrates reasoning and action generation within a single interaction loop. Instead of directly predicting an action from the observation, the model first produces a short reasoning step describing its interpretation of the environment state, followed by the selected action. This intermediate reasoning encourages the model to analyze the current situation before deciding on the next move. Each step therefore follows the structure *Thought* → *Action*, where the Thought explains the reasoning and the Action specifies the navigation command.

In our implementation, the MiniGrid environment state is converted into a structured textual observation describing the agent's position, orientation, nearby tiles, and goal direction. This observation, together with environment rules and recent reasoning history, is included in the prompt provided to the LLM. The model then outputs a Thought–Action pair, from which the action (`turn_left`, `turn_right`, or `move_forward`) is extracted and executed in the environment. The resulting observation is appended to the prompt history and the interaction continues until the goal is reached or the episode terminates.

3.4 Tree of Thought

Unlike single-path reasoning strategies such as Chain-of-Thought[9], the Tree-of-Thought (ToT) agent [10] explores multiple candidate actions at each timestep and selects the most promising one through explicit evaluation and pruning. At each step, the agent enumerates all available actions (e.g., `turn_left`, `turn_right`, `move_forward`) and treats each as a branch in a shallow reasoning tree. For every candidate, the LLM is prompted to imagine the resulting state, estimating the new agent position, orientation, and surrounding tile layout based solely on the current observation and the proposed action.

Each imagined successor state is then evaluated by the LLM, which scores branches based on estimated goal proximity, perceived hazards such as lava tiles and walls, and overall progress toward task completion. Branches deemed unsafe or unproductive are pruned, and the agent executes the highest-scoring action in the environment. The resulting observation is fed back into the next iteration of the loop. Because both imagination and evaluation are handled entirely by the LLM, the ToT agent operates as a depth-2 lookahead search driven by language-based reasoning.

3.5 Buffer of Thought

Buffer of Thought (BoT) augments action selection with a compact strategy memory rather than full multi-branch search like Tree-of-Thought [10]. Following the idea of reusable thought templates proposed by Yang et al. [9], our implementation maintains a small meta-buffer of navigation templates (e.g., direct navigation, obstacle avoidance, lava crossing, and recovery). At each timestep, the observation is first distilled into structured features (agent position, orientation, nearby occupancy, goal direction, and hazards). The meta-buffer then retrieves the most suitable template based on the current context, including whether the agent is blocked, in a repeated state, or likely to reduce goal distance by moving forward.

Conditioned on the selected template, the agent forms a ranked list of candidate actions and prompts the LLM to choose one action from this restricted set. A hard safety rule is applied afterward: if the front cell is unsafe (wall or lava), `move_forward` is overridden by a safe turning action. This design keeps per-step reasoning lightweight while preserving context-sensitive behavior.

In addition, BoT includes post-episode adaptation. When an episode fails, we classify a failure signature (e.g., lava death, timeout, or looping) together with a coarse grid category. If the same signature repeats, the agent asks the LLM to synthesize a one-sentence strategy template from recent trajectory traces and adds it to the meta-buffer under deduplication and capacity limits. Compared with Tree-of-Thought-style explicit branching, this template-retrieval mechanism is cheaper at inference time when the seed templates are well crafted and relevant. If the agent has not explored enough or has not yet accumulated useful templates, performance can be weaker until more effective templates are built over time [9].

4 Experiments

4.1 Experimental Setup and Hyperparameters

For all experiments, we use Qwen2.5-7B-Instruct without quantization. To ensure fair comparison, all experiments use the same seed, temperature, and status prompt. Specifically, we set the seed equal to the episode number, since a fixed seed would result in the environment always being the same, and temperature to 0 to ensure determinism. All experiments calculate the average metrics across 20 episodes, with a maximum allowed steps of 100 per episode and no limit on the number of tokens the agent may use.

All prompting strategies are compared to the baseline that simply asks for the model to predict an action. The first evaluation metric is the Success Rate (SR), which measures how often the agent reaches the goal in the environment. To quantify efficiency, we also measure the average steps taken to reach the goal and the average tokens (input and output) used to take an action. Finally, we quantify the cost efficiency using the average steps taken to reach the goal.

4.2 Experimental Result

Table 1 shows our results on the Empty8x8 environment, Table 2 the results on LavaGapS6, and finally Table 3 the results on LavaCrossingS9N2.

Table 1: Performance comparison on MiniGrid-Empty-8x8-v0 (Qwen2.5-7B-Instruct)

Method	Success Rate (SR)	Steps	Tokens per Step	Tokens per Rollout
Baseline	100.00%	11.60	192.57	2233.81
ReAct	90.00%	46.33	378.52	17536.83
Chain of Thought	100.00%	16.60	617.50	10250.5
Tree of Thought	100.00%	11.00	2817.58	30993.38
Buffer of Thought	100.00%	11.00	207.45	2281.95

Table 2: Performance comparison on MiniGrid-LavaGapS6-v0 (Qwen2.5-7B-Instruct)

Method	Success Rate (SR)	Steps	Tokens per Step	Tokens per Rollout
Baseline	40.00%	21.62	206.47	4463.88
ReAct	35.00%	23.57	400.21	9432.95
Chain of Thought	45.00%	15.78	670.30	10577.33
Tree of Thought	50.00%	10.33	2200.56	22731.78
Buffer of Thought	55.00%	49.15	229.32	11271.08

Table 3: Performance comparison on MiniGrid-LavaCrossingS9N2-v0 (Qwen2.5-7B-Instruct)

Method	Success Rate (SR)	Steps	Tokens per Step	Tokens per Rollout
Baseline	5.00%	35.00	249.11	8718.85
ReAct	0.00%	—	458.34	—
Chain of Thought	15.00%	41.67	666.30	27764.72
Tree of Thought	20.00%	30.67	3466.50	106317.56
Buffer of Thought	10.00%	91.30	274.6	25070.98

Nearly all agents achieve a perfect success rate on the easiest environment, Empty8x8. For LavaGapS6, on the other hand, success rates hover around 40-55%, and on the hardest environment, LavaCrossingS9N2, even the best method achieves only a 20% success rate. The difficulty of this final environment is due to various factors. The abundance of lava compared to LavaGapS6 increases the number of opportunities for the agent to walk in to lava. The larger grid also makes it so that the agent cannot locate the goal as easily, requiring more exploration.

Overall, it can be found that the reasoning-enhanced LLM agents outperform the baseline Reactive agent with higher success rate and less exploration steps. **However, it should be noted that the performance of the ReAct agent is slightly worse than the Reactive one both on success rate and exploration steps.** This might be due to the limited capabilities of a 7B model. When trying to predict thought and action at the same time, it worsens the performance as compared to predicting just the action, as the model now has much more to get right at the same time for a less expressive model within just a single reasoning step as opposed to Chain-of-Thought. We hypothesize that using a bigger model would improve ReAct's performance as compared to the Reactive baseline.

On the other hand, Tree of Thought and Buffer of Thought achieve the best performance overall across the three environments, but with performance tradeoffs. Tree of Thought uses far more tokens than the other methods, while Buffer of Thought takes many more steps, due to its exploratory approach. This underlines the fact that the methods all present the user with a tradeoff, where more sophisticated methods can give better results but at the cost of reduced efficiency. Notably, the simple baseline method consistently uses the least tokens compared to any other method. This makes it the most efficient option in terms of tokens, but at the expense of performance. Fig. 6 shows which methods are Pareto-optimal for each environment, with baseline and Buffer of Thought being the only two that are optimal in all environments, meaning that regardless of environment, no other method is objectively superior to these in both success rate and token efficiency.

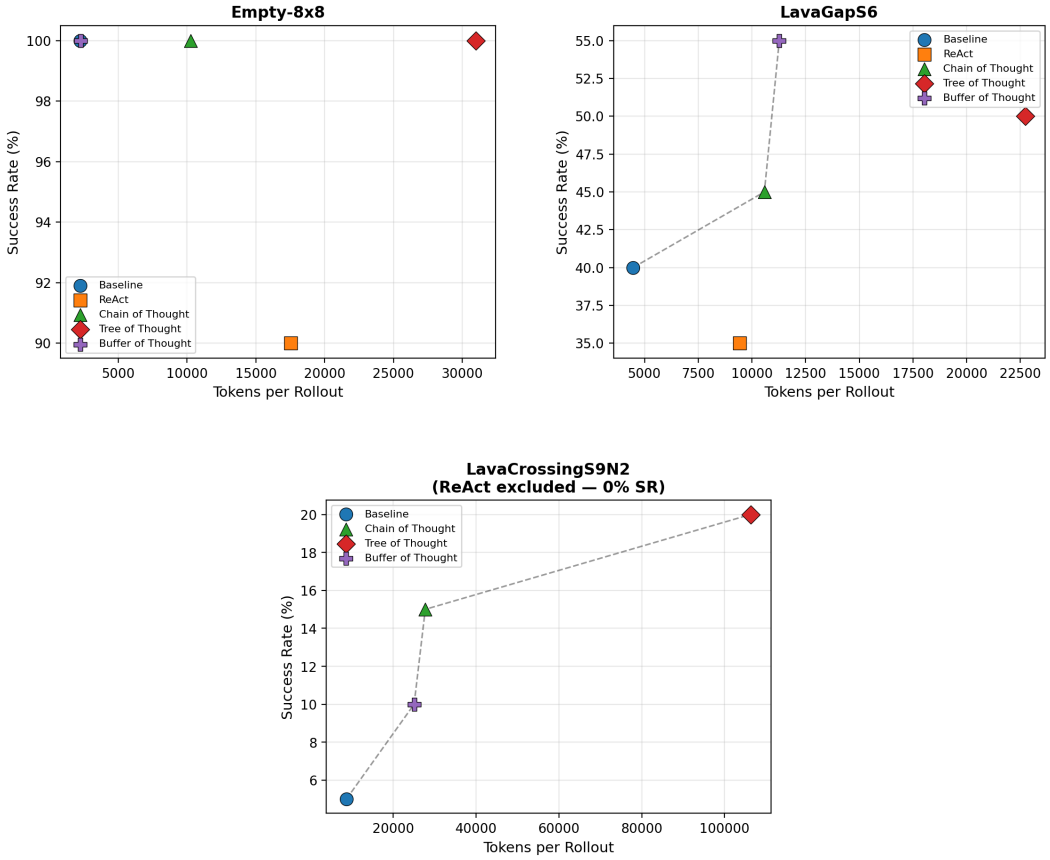


Figure 6: Pareto frontier of success rate vs. token cost per rollout across three MiniGrid environments using Qwen2.5-7B-Instruct. The dashed line connects Pareto-optimal methods.

5 Related Work

5.1 LLM-Based Autonomous Agents

The deployment of large language models as autonomous agents capable of reasoning, planning, and acting in interactive environments has become a rapidly growing research direction. [7] provides a unified framework decomposing LLM-based agents into profile, memory, planning, and action modules, and distinguish between single-path and multi-path reasoning strategies for the planning module. Our work directly instantiates this taxonomy: we compare five agents that span the spectrum from no explicit planning (Reactive) through single-path reasoning (CoT, ReAct) to multi-path search (ToT) and memory-augmented reasoning (BoT), all within a controlled navigation setting.

5.2 LLM Agents in Grid Navigation

Grid-world environments have served as a common testbed for evaluating LLM agent capabilities. Tang et al. [6] proposed WorldCoder, a model-based agent that builds an explicit world model as a Python program through environment interaction, and evaluated it on MiniGrid and Sokoban. WorldCoder demonstrated superior sample efficiency over deep RL and compute efficiency over ReAct-style agents by front-loading LLM calls to synthesize a reusable world model. In contrast to WorldCoder's model-based code-synthesis approach, our study focuses on comparing prompting-only strategies where all reasoning occurs through structured LLM inference without learning or updating an external model, providing a complementary perspective on how different inference-time reasoning frameworks perform in navigation tasks.

We evaluate all five agent strategies on the MiniGrid environment [1], a lightweight suite of discrete grid-world tasks with partially observable 2D navigation, configurable obstacles, sparse rewards, and natural-language mission strings.

6 Conclusion

In this work, we conducted a controlled comparison of five LLM agent strategies for navigation in MiniGrid: a reactive baseline, ReAct, Chain of Thought, Tree of Thought, and Buffer of Thought. Across three environments of increasing difficulty, our results show that explicit reasoning or planning can improve navigation performance over purely reactive action selection, especially as the task becomes more challenging. However, these gains are not free as stronger reasoning methods generally incur higher token cost, longer inference time, or initialization overhead.

We observe that all methods perform strongly on the simplest environment Minigrid-Empty-8x8-v0, suggesting that sophisticated planning is not necessary for easier goal-reaching tasks. As the environment becomes more hazardous and exploration-heavy, the differences between prompting strategies become more pronounced. Chain-of-Thought (CoT) [8] provides a good middle ground, improving capability while avoiding the overheads associated with Buffer-of-Thought (BoT) and Tree-of-Thought (ToT). Tree-of-Thought [10] and Buffer-of-Thought [9] achieve the best performance overall on the harder settings, but they do so with different tradeoffs. ToT is the most expensive in token usage because it evaluates multiple candidate reasoning branches at each step, while BoT can remain comparatively token-efficient but often requires more exploratory steps. By contrast, the reactive baseline remains the cheapest strategy overall in terms of tokens, but it is less reliable once the navigation problem requires longer-horizon reasoning.

Overall, our study suggests that there is no universally best LLM agent strategy for all navigation tasks. The choice method depends on the deployment constraint: if token budget and latency are critical, simpler reactive or CoT-based agents may be preferable; if maximizing task success is the priority, more structured planning methods such as ToT or memory-augmented methods such as BoT become more attractive. Future work can extend this comparison to larger and more capable models, additional MiniGrid [1] tasks involving keys, doors, such as DoorKey, and object manipulation, such as Blocked Unblocked Pickup, and possible hybrid agent designs that may combine deliberative planning with reusable memory. It would also be useful to study methods such as quantization and prompt compression to reduce latency and cost while preserving the benefits of more advanced reasoning strategies.

7 Use of LLM Tools

We used GPT-based tools as development assistants for selected parts of the project workflow. Their use was limited to support tasks such as drafting and refining prompt wording, improving code, and identifying potential implementation bugs.

All experimental design choices, implementation logic, result generation, and final technical decisions were made and verified by us.

8 Group Member Contributions

Table 4: Team member contributions

Group member	Contribution
Changxin Wan	Basic testing framework, Reactive LLM agent
Calvin Glisson	Expanded testing framework, CoT agent
Kedar Nitin Shiralkar	Environment Wrapper Base, ReAct Agent Code
Bhargav Srikanta Prasad Hoskote	Environment Wrapper advancement, ToT agent
Vedant Deepak Borkute	Structured output framework, BoT agent

References

[1] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo de Lazcano, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid miniworld: Modular customizable reinforcement learning environments for goal-oriented tasks. *arXiv preprint arXiv:2306.13831*, 2023. URL `https://arxiv.org/abs/2306.13831`.

[2] Sihao Hu, Tiansheng Huang, Gaowen Liu, Ramana Rao Kompella, Fatih Ilhan, Selim Furkan Tekin, Yichang Xu, Zachary Yahn, and Ling Liu. A survey on large language model-based game agents. *arXiv preprint arXiv:2404.02039*, 2024.

[3] Narjes Nourzad and Carlee Joe-Wong. Mira: Memory-integrated reinforcement learning agent with limited llm guidance. *arXiv preprint arXiv:2602.17930*, 2026.

[4] Ihor Omelchenko and Volodymyr Strukov. Reflective memory architecture for adaptive planning in hierarchical llm agents in virtual environments. *Bulletin of VN Karazin Kharkiv National University, series «Mathematical modeling. Information technology. Automated control systems»*, 68:62–69, 2025.

[5] Byeong-Ju Park, Sung-Jung Yong, Hyun-Seo Hwang, and Il-Young Moon. Optimizing agent behavior in the minigrid environment using reinforcement learning based on large language models. *Applied Sciences*, 15(4):1860, 2025.

[6] Hao Tang, Darren Ni, Jason Kamber, and David C. Parkes. Worldcoder, a model-based llm agent: Building world models by writing and retrieving code. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL `https://arxiv.org/abs/2402.18154`.

[7] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2024. URL `https://arxiv.org/abs/2308.11432`.

[8] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL `https://arxiv.org/abs/2201.11903`.

[9] Ling Yang, Zhaochen Yu, Tianjun Zhang, Shiyi Cao, Minkai Xu, Wentao Zhang, Joseph E. Gonzalez, and Bin Cui. Buffer of thoughts: Thought-augmented reasoning with large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL `https://arxiv.org/abs/2406.04271`.

[10] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.10601.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.